

Climbing app

My project is a chatbot that allows users to find the closest and most suitable climbing route. The project is written in TypeScript and in its code is fetching Ollama LLM and OpenBeta API. The TypeScript code is using the Xstate library to perform dialogue with the user.

The Open Beta website is an open-source project that uses user-provided data. It can be used as a website with climbing information, and it has its own API published on GitHub that I'm using in my code. The idea for authors was to gather betas about climbing routes from all over the world. Beta is information about how to ascend a climbing route and the specific climbing techniques needed for the challenge. So also my first idea of the project was to work with betas datasets. Later on, while I was working with available data, I found out that there are very few betas available on the website. Hence, I focused on data that is available for each climbing route, which is its location and climbing grade. Open Beta API gathers information about different climbing areas, mostly in the USA, which is the last part of the dialogue with the chatbot. Because there are no resources with this exact dialogue structure, I created the model Dialogues myself:

- Hey. Where would you like to climb?
- I would like to climb in California
- Where are you located right now?
- I'm in LA
- What grade do you usually climb?
- In the gyms, I mostly climb V5s
- You should check out this Master of Reality boulder

Open Beta is based on the GraphQL language. With the help of this language, we can fetch a query to the API. The data chatbot wants to get from the Open Beta API is information on all climbs in a particular region. The area is chosen by the user in the first question. The *area* type in the open beta data set is recursive, meaning that the USA is an area, as are California and Yosemite Valley. Each area can have an ancestor of two data types: climbs or area, hence the number of areas in the whole data set is enormous. I could have forced LLM to choose from a list of areas, but the list would have contained dozens of thousands of objects. To make it more efficient, I chose only one area that the user can choose (Yosemite Valley). The

method by which the LLM matches the user's answer to objects in the list will be used in the 3rd question.

All of the user's utterances and the LLM's internal instructions and utterances are stored in lists of objects of the *message* type. The main list is called *Messages*. When a user answers a question “in which area would you like to climb”, their answer is added to the main list *messages* and also to the secondary list *classificationMessages*. The main *messages* list stores the history of the whole conversation. The *classificationMessages* list role is only to detect if a user wants to climb in Yosemite Valley. In this particular list, LLM will generate classification messages. Content of classification: Messages can look like this:

1. {role: 'system', content: 'You are a strict boolean classifier. Evaluate the user's last utterance based on this rule: "Does the user want to climb in Yosemite Valley?"
Return ONLY valid JSON: {"result": true} or {"result": false} Do NOT include any other text.'}
2. {role: 'user', content: 'I would like to climb in Yosemite Valley'}
3. {role: 'assistant', content: '{"result": true}'}

If the user wants to climb in only one possible area, the chatbot will continue. Otherwise, the chatbot will try to persuade users to choose Yosemite Valley. If no user's utterance is detected, the chatbot will ask the user to say something. An API is used to fetch all climbing data for a particular climbing area. The next step in the code is to restructure the data into a dictionary with boulder names as keys and data such as location and grade in the value list.

The second question will try to gather information about the user's location. LLM will be asked to provide the latitude and longitude of this particular location. This action will be performed again in a separate list of *message* objects, this time called: *latlongMessages*. LLM should give back the lat and long in an easy-to-detect format. Then the function *stringToCoordinates* will extract the LLM's answer, including lat and long, into a list of integers using regex. Now, when a chatbot knows where all the boulders are located as well as the user's location, it can count the distance between each user and boulder. This is done simply using Pythagoras' theorem. Later, all climbs are sorted by the user's destination (in the *find_closest_climb* function).

In the last question, the chatbot will try to determine the user's climbing grade. To prepare for this task, the programme will gather all accessible climbing grades in a particular area and

compile them into a list (*make_all_grades_list* function). This will be stored in *context*. Again, in a separate list of *message* objects called *gradeMessages*, LLM will need to detect which climbing grade the user mentioned in their utterance. Because we want the LLM to give a precise answer, we will provide it with an already-created list of all climbs. In the last step, the code will search a list of climbs to find the closest boulder with this grade (*find_matching_grade* function). This function allows choosing a grade that is more or less similar, not too precise. The desired climb is printed on the screen and should be presented in the last chatbot's utterance.

The model does not always perform perfectly because of hallucinations. Most often, an LLM gets confused by roles like assistant, system, and user. Sometimes, instead of greeting users in the beginning, it produces an instruction on how to greet a potential user. It's because the message is produced in the role of the system instead of an assistant. I chose this simply because within the assistant role system wasn't working at all. The same when generating a message in case of *ASR_NOINPUT*, the silent user needed to be introduced, cause no other configuration worked.

In the future, this dialogue system could be applied to more areas. Also, in the current version, the user could have answered the question about climbing grades with a word like "apple," and the system is still forced to interpret this as one of the climbing grades. This should not be allowed in a more advanced version of this app. To keep it entertaining, the conversation between the chatbot and the user could continue. For example, about other boulders nearby.

Stanislaw Lorys